COMP 1039 – Design Patterns with C++

Assignment 1: Dead Man's Draw++

Weighting: 20%,
Due Date: See course website

Version	Date	Notes
1.0	2 Mar 26	Initial release

Introduction

In this individual assignment, you have been contracted by a mysterious client with a passion for pirate lore to develop a command-line based game called **Dead Man's Draw++** in C++. The client insists this is a wholly original idea, though keen observers might notice its uncanny resemblance to *Dead Man's Draw*, a fast-paced card game of luck, strategy, and risk-taking from Mayday Games.

In **Dead Man's Draw++**, 2 players take turns drawing cards from a shared deck, each card representing a different type of pirate-themed loot—such as Cannons, Maps, Keys, Chests, and more. The goal is to collect the highest total score by the end of the game. Each turn, a player can continue to draw cards to increase their haul—but if they draw a card with the same suit twice in a single turn, they bust and gain nothing. However, each suit has a special ability that can influence the flow of the game, allowing for strategic plays and dramatic reversals.

The game is played over a fixed number of turns (or until the deck runs out), with players alternating turns and accumulating treasure based on the cards successfully banked. Your task is to implement the full game logic in C++, using object-oriented programming principles, appropriate data structures, and clear, maintainable code.

Will you play it safe—or push your luck? It's time to code your way to pirate glory in **Dead Man's Draw++**.

Learning outcomes

After completing this assignment, you will have learnt to:

- Create a basic system in C++ using an object-oriented design.
- Write C++ code using modern C++ memory management.
- Write safe C++ code.
- Use C++ collections.

Assignment and submission requirements

The assignment has the following requirements. Failing to address any of these requirements will result in deducted marks.

1. The assignment must be submitted via LearnOnline.
2. You must submit three files: 1) a zipped GIT repository, 2) a functioning executable (.EXE), and 3) the system design document.
3. These files must be submitted as three separate files to LearnOnline.
4. The GIT repository must:
    a. Include the full GIT history.
    b. Contain your Visual Studio 2022 Solution and project files.
    c. It must build and run from Visual Studio 2022.
    d. The repository must be compressed using the common ZIP compression.
5. The functioning executable must:
    a. Run from command line under Windows 10 or higher.
6. The system design document must:
    a. Be submitted as PDF format.
    b. It must contain your UML diagram.
    c. It must contain any design decision notes that you make during your design.
    d. Note: it may also exist as Word document or similar in your repository.
    e. Note: it may be updated during your implementation as your design will likely change and evolve.

Game rules

Dead Man's Draw++ is a competitive, turn-based card game for 2 players. Players take turns drawing cards from a shared deck and placing them into their **Play Area**, trying to collect the most treasure (points) by the end of the game. Each card has a *suit* (e.g., Key, Chest, Cannon) and a *point value*. Players must decide when to stop drawing and **Bank** their loot—or push their luck and risk losing it all.

Deck and Cards

- The **Deck** contains **54 cards**, with **9 different suits**, each suit having 6 cards with different **point values**.
- Each card has:
    - A **suit** (e.g., Anchor, Chest, Key, Sword, etc.)
    - A **point value** (e.g., 2–7, depending on suit distribution)
    - A **special ability** (see *Suit Abilities* below)

Turn Structure

On a player's turn:

1. They draw a card from the shared **Deck** and place it in their **Play Area**.
2. The player may **continue drawing**, one card at a time, or **stop** at any point to collect their loot.
3. When they draw a card, they must play its **Special Ability**.

4. If the player draws a card with a **suit that has already been drawn this turn**, they **bust**—all cards in the play area are discarded to a shared **Discard Pile** and the turn ends.
5. If the player **stops** voluntarily before they **bust**, they move all cards in the **Play Area** to their **Bank** (permanent collection) which is scored at the end of the game.
6. The turn then passes to the other player.

Scoring

- At the end of the game, each player's score is the **sum of the highest point card in each suit** they have banked.

Game End

- The game ends when the deck is empty or a set number of turns have been played (e.g., 20 total turns).
- The player with the **highest total score** wins.

Card Abilities

Each type of card (i.e. suit) has a different ability that must be played when drawn. These abilities are listed in Table 1. The ability *must be played* even if the results of that ability means that the player will **bust** for that turn. Many abilities require drawing from the Bank or the Discard Pile. If there are no cards to draw from in these cases, play continues with a message saying that there were no cards to draw.

Table 1: The types of cards available in Dead Man's Draw++ and the number that appear in the deck

Suit	Ability Description	Point Values
Cannon	Discard the top card (i.e. the highest value) of any suit from the other player's Bank to the Discard Pile.	2,3,4,5,6,7
Chest	No immediate effect. If banked with a Key card, draw as many bonus cards from the Discard pile as you moved into your Bank.	2,3,4,5,6,7
Key	See above (Chest).	2,3,4,5,6,7
Sword	Steal the top card (i.e. the highest value) of any suit from the other player's Bank into your Play Area. You must select one card.	2,3,4,5,6,7
Hook	Play the top card (i.e. the highest value) of any suit from your Bank into your play area. You must select one card.	2,3,4,5,6,7
Oracle	Peek at the top card of the deck before choosing whether to draw	2,3,4,5,6,7
Map	Draw 3 cards from discard pile. You must play one of the cards drawn into your play area.	2,3,4,5,6,7
Mermaid	No ability but the cards have a higher point value	4,5,6,7,8,9
Kraken	Must draw and play three cards consecutively.	2,3,4,5,6,7

User interface screens

The following subsections demonstrate the expected user interface for the system. Your user interface must match this output exactly except for the random selection of cards. Further examples of output are provided in Appendix 1.

Game start

```
------                 _   ___  ___             _
|  _  \               | |  |  \/  |            ( )
| | | | ___   __ _   __| |  | .  . |  __ _  _ __ |/ ___
| | | |/ _ \ / _` | / _` |  | |\/| | / _` || '_ \  / __|
| |/ /|  __/| (_| || (_| |  | |  | || (_| || | | | \__ \
_____ \___| \__,_| \__,_|  \_|  |_/ \__,_||_| |_| |___/
|  _  \                       _     _
| | | | _ __  __ _ __      __ _| |_  _| |_
| | | || '__|/ _` |\ \ /\ / /|_   _||_   _|
| |/ / | |  | (_| | \ V  V /   |_|    |_|
|___/  |_|   \__,_|  \_/\_/

Starting Dead Man's Draw++!
--- Round 1, Turn 1 ---
Sasha's turn.
Sasha's Bank:
| Score: 0
Sasha draws a Map(3)
    No cards in the discard. Play continues.
Sasha's Play Area:
    Map(3)

Draw again? (y/n): |
```

A game Turn starts by announcing the Round and Turn number. The player's name is presented as well as the Cards in their **Bank**. At the very start of the game, both player's banks are empty. The game then draws a card. If the player does not **bust** (i.e. have a card of the same **Suit** already in the **Play Area**) then the game executes the ability of the card.

In the above example, the card is a Map, which does nothing when no cards have been previously discarded.

Assuming the player did not bust, then the game prints the cards in the current play area and asks whether to "Draw again? (y/n): ".

Players select whether they want to continue drawing from the **Deck**.

The prompt should:

- Read a string from the user.
- If the user typed "y", another card is draw.

Continuing their turn

```
Draw again? (y/n): y
Sasha draws a Hook(2)
    No cards in your Bank. Play continues.
Sasha's Play Area:
    Hook(2)
    Map(3)

Draw again? (y/n):
```

If the player continues their turn, the game draws a new card and executes the ability of the card.

Bust

```
Marge's Play Area:
    Key(2)
    Sword(3)
    Hook(6)
    Oracle(6)
    Mermaid(4)
    Kraken(6)

Draw again? (y/n): y
Marge draws a Oracle(7)
BUST! Marge loses all cards in play area.
```

If a player draws a **Card** of a **Suit** they already have in the **Play Area**, the player is **bust** for that turn. The ability of the card **does not** execute. All cards in the Play Area go directly to the Discard Pile and play switches to the other player.

In the above example, the player has an Oracle(6) in their play area. They draw another Oracle (Oracle(7)). This causes the player to Bust.

Banking

If a player chooses not to draw again, all cards in their Play Area move to their Bank. The player's bank is printed in a similar format to the **Play Area**. Their total score, being the sum of the highest value cards in each Suit, is also shown.

```
Draw again? (y/n): n
Marge's Bank:
    Hook(5)
    Oracle(5) Oracle(3)
    Mermaid(6)
| Score: 16
```

In the above example, the player has three unique suits. The highest values of each suit are Hook(5), Oracle(5), Mermaid(6), producing a current score of 16.

Printing Banks and Play Areas Collections

Bank and play areas should be printed using the same method. Each line should be for a suit in the collection, and each card should be sorted from highest value to lowest value.

```
Sasha's Bank:
    Cannon(5)
    Chest(5)
    Key(6)
    Sword(3) Sword(2)
    Hook(7) Hook(6) Hook(3)
    Map(4)
    Mermaid(9) Mermaid(6) Mermaid(4)
    Kraken(7)
| Score: 46
```

Required classes

Your assignment must design and implement at least the following classes:

Game — The Game object drives the game. The Game object:

- Is responsible for initialising and starting a new game.
- Is responsible for ending a game and printing final scores.
- Is responsible for creating the card deck.
- Is responsible for shuffling the deck.
- Is responsible for initialising players.
- Is responsible for controlling a turn in the game:
    - Drawing a card.
    - Asking if the player wants to draw another card.
    - Handing the game to the other player if the current player busts.
- Knows the current turn and round of the game.
- Knows the current player.

There should only be a single instance of Game at any one time.

Player — Represents a player in the game. A player object:

- Is responsible for managing and adding cards to the player's current Play Area.
- Is responsible for knowing if the player's current play area is bust (i.e. there are two of the same type of card in the play area).
- Is responsible for managing the player's Bank and moving cards from the play area to the bank.
- Is responsible for printing the player's play area and bank to the screen.
- Knows the player's current total score.
- Knows the player's name, randomly initialised from a fixed list.

When implementing player, we suggest having a function that adds a card to the player's play area and removes it from the deck:

- ```
  // plays the card into the play area
  // returns true if this causes the player to bust,
  // otherwise the card's ability is performed and this function returns false
  bool playCard(Card *card, Game& game);
  ```

When selecting the name for the player, select it from a random list:

- ```
  std::string names[] = {"Sam", "Billy", "Jen", "Bob", "Sally", "Joe", "Sue",
  "Sasha", "Tina", "Marge"};
  ```
- ```
  _name = names[rand() % 10];
  ```

Card — Represents the base type of a card in the game. The Card class should be an abstract class that other types of card classes (e.g. Cannon, Chest, Key, Sword, Hook, Oracle, Map, Mermaid, Kraken) inherit from. A card should:

- Contain an enum for the type of card it is.
- Is responsible for presenting the card type (i.e. printing the card type to screen) back to the player.
- Is responsible for playing that type of card.

When implementing a card, define an enum for the type of card it represents:
  ```

- `enum CardType {Cannon, Chest, Key, Anchor, Sword, Hook, Oracle, Map, Mermaid, Kraken};`

Add a function that returns this type:

- `const CardType& type() const`

Add a function that returns the type of card and value as a string (e.g. “Oracle(7)”):

- `virtual std::string str() const = 0;`

The Card is responsible for executing the ability of the Card. That way, Game and Card are loosely coupled, and it is easier to add or remove cards as needed. To enable this, we suggest having the following function in Card:

- ```
  // play the effect of this card
  virtual void play(Game &game, Player &player) = 0;
  ```

Override this function to handle each type of cards’ ability. There is one “type” of card (the Chest and the Key together), where the ability occurs only when the player banks that card. You will need to think carefully about the order of operations here. We recommend having a function in Card with the following signature, called just before being added to the bank, to handle this:

- `virtual void willAddToBank(Game &game, Player &player) { }`

We also recommend type def-ing a card collection:

- `typedef std::vector<Card*> CardCollection;`

Hints and tips

Pseudocode. A lot of the challenge in the assignment is due to the complexity of operations. The order in which things occur matters and requires careful thought. Somethings to consider:

- A card only plays its ability if the player has not busted. This requires the game to stop if the player has busted before playing the card’s ability.
- Some abilities require selecting a card from the Discard Pile or a player’s Bank, adding that card to the Play Area. This may cause the player to bust. If the player does not bust, that card’s ability *must be played*.
- Chest and Key are unusual. Their ability only occurs when the player banks the cards in their play area, and only if the player has both the Chest and the Key.

All these considerations should be planned out. Pseudocode is your best tool here.

Game round scoring. The game object should do end of round scoring. However, each player object should be responsible for actually calculating the total score of their Bank.

This is slightly tricky as you must add the value of the highest card in each Suit. Do not simply add all the cards together. The following is suggested pseudocode for scoring.

```
Create a map S of scored cards
For each card c in the players tableau
    Let t be the type of card c
    If value of t in S is less than the value of c
        Set the value of t in S to be the value of c
Let total be 0
For each value in S
```
```

```
Total = total + value
```

Shuffling the deck. The following code will shuffle a deck *in-place*:

```
void shuffleDeck(CardCollection& cards) {
    CardCollection shuffleDeck { cards.begin(), cards.end() };
    std::shuffle(shuffleDeck.begin(), shuffleDeck.end(), std::mt19937{std::random_device{}()});
    std::copy(shuffleDeck.begin(), shuffleDeck.end(), cards.begin());
}
```

Task 1 – System design

> *Refer to your previous courses (e.g. OOP, Systems Design…) that have covered UML for the skills necessary to complete this task.*

This task requires you to create UML class diagram of the system to be built. *This will help you think about your design before you start building it!* Use the description of the classes above and the domain model to inform the class diagram. There may be more classes in the system than those described above.

1.1. Create a document to describe your system design.
   - The document must include a title, your full name, email address, and student ID.

1.2. Create a GIT repository and add your document to the repository.
1.3. Create the UML class diagram for the system.
1.4. Document any design decisions you have made as dot points (e.g. why certain classes have certain methods).
1.5. Add a PDF of your UML class diagram to your repository.
1.6. Commit your changes to your repository.

Make sure to:

- Carefully consider the direction of the associations between classes. Only have a class accessible by another class if it is necessary.
- Indicate multiplicity on the associations.

Task 2 – Implementation

> *Weeks 1–5 of the course focusses on the knowledge and skills necessary to complete this task.*

2.1. Create a new C++ console application using Visual Studio 2022 in your repository.
2.2. Add an appropriate GIT ignore file.
2.3. Add the provided source code available on the course website.
2.4. Commit your changes to GIT.
2.5. As a sanity check, clone your repository somewhere else on your computer and ensure the entire Visual Studio solution is there as expected.
2.6. Implement the system based on the class diagram you design.
2.7. Commit your changes to GIT.

[Optional] Bonus Task – Algorithms

This task is optional for bonus marks. You can only get a maximum of 100% on the assignment, but some students may use this task to maximise their marks.

3.1. Implement the Anchor suit.

Suit	Ability Description	Point Values
Anchor	Any cards draw before the Anchor card are safe and moved to the player's Bank, even if the player busts.	2,3,4,5,6,7

This is a complicated suit. Essentially it breaks the usual flow of the game. If you want these bonus marks, you must implement this suit without putting the behaviour in the Game class; the behaviour should stay in the AnchorCard class.

Design constraints

The following design constraints must be met throughout the assignment:

DC1. The required classes specified above must be implemented.
DC2. The Visual Studio 2022 solution you submit must compile and run.
DC3. C++ standard library collections must be used for storing collections of objects.
DC4. You must dynamically allocate your Card and Player objects (i.e. use pointers).
DC5. Dynamically allocated memory must be cleaned up correctly.
DC6. Collections of pointers must be cleaned up correctly.
DC7. Class member functions should be declared const unless they cannot be.
DC8. Initialiser lists must be used for constructors where initialisation is required.
DC9. The code style guide available on the course website[1] must be followed with the following exception: you can decide whether to place opening braces on the statement line or the following line.
DC10. Your system must not use external libraries (e.g. BOOST).

Workplan

The following workplan is suggestion. First, do not wait until the due date to start the assignment; under the "GIT" marking criteria, we are expecting to see consistent commits **each week**. You can progressively build the system as your understanding of C++ develops. Build the classes first, refactor the collections and memory management, add const declarations. *The design of C++ allows this progress refactoring of the system and is your best approach for a successful assignment*.

- Make sure to watch a rules summary of the real Dead Man's Draw game: (How To Play Dead Man's Draw). The game you are implementing has been simplified to be achievable by:
  - Removing the Anchor suit,
  - Removing Characters or any other additions, and
  - Making it 2 players only.
- You have enough information to immediately begin drawing the system design diagram (Task 1). *When done first, the system design diagram will actually help you understand, design, and write your code.*
- Create "skeleton" classes for your project based on your system design diagram.

[1] https://lo.unisa.edu.au/course/view.php?id=33480&sectionid=931464

- For the implementation task (Task 2):
    - During Week 3, you have enough information to build the basic class structures.
    - During Week 4, you will have enough information to implement the C++ collections.
    - During Week 5, you will have enough information to implement modern memory management.
    - Week 6 should then be spent testing and submission.

Constantly commit to GIT. Use it as a safety net so that you can always revert to a working version of your project.

Marking Scheme

Criteria	Mark
Design (15%)	
Design diagram – Marked on completeness of diagram and syntax. Required classes are present and used correctly.	15%
Functional requirements (55%)	
Gameplay loop — The gameplay loop (starting a game, drawing a card, busting, switching players, winning) is handled correctly.	10%
Card abilities — Card abilities are handled correctly.	40%
Scoring — Scores are calculated correctly.	10%
Non-functional requirements (30–40%)	
GIT – GIT has been used. Optimal marks are rewarded for 1) at least one commit each in Week 4, Week 5, Week 6 and 2) at least one commit per functionality in the system. Commit messages should use the imperative mood.	10%
Design constraints – The design constraints have been addressed in the code implementation.	15%
[Optional] Bonus task (+10%)	
Bonus task – The bonus task has been sufficiently addressed.	+10%

- The maximum marks achievable is 100%.
- You will be required to explain your design and code during a practical class. Not being able to explain any of these elements can result in losing marks.

Penalties

You will be expected to be able to understand, explain your code, and adapt it. This will require demonstration, explanation, and adaption during one of your practical classes after the assignment is submitted. Not being able to adequately demonstrate, explain, or adapt your code can result in a **deduction of marks**.

Extensions

Late submissions will not be accepted for this course unless an extension has been approved by the course coordinator. Late submissions that have not been approved will receive a mark of zero. Refer to the course outline for further information regarding extensions.

Academic Misconduct

This is an individual assignment. Your submitted files will be checked against other student's submissions, and other sources, for instances of plagiarism.

Appendix 1 — Example output

```
_____                  _   ___ ___              _
|  _  \                | |  |  \/  |             (_)
| | | |___ __ __ __ __| |  | .  . |___ _ __ ___  _ __ ___
| | | |/ __| '__| '__| |  | |\/| |/ _` | '__/ _ \| | '__/ _ \
| |/ /| (__| | | | | | |  | |  | | (_| | | | (_) | | | | (_) |
|___/ \___|_| |_| |_|_|  \_|  |_/\__,_|_| \___/|_|_| |_| \___/
_____                                  _ _   _
|  _  \                                | | | | |
| | | |___ __ __ __ __ __ __ __ __ __ __| | | | |
| | | |/ __| '__| '__| '__| '__| '__| '__| | | | |
| |/ /| (__| | | | | | | | | | | | | | | | | |
|___/ \___|_| |_| |_|_|_|_|_|_|_|_|_|_|_|_|_|_|_|

Starting Dead Man's Draw++!
--- Round 1, Turn 1 ---
Sasha's turn.
Sasha's Bank:
| Score: 0
Sasha draws a Mermaid(5)
 No effect but Mermaids are worth more.
Sasha's Play Area:
 Mermaid(5)

Draw again? (y/n): y
Sasha draws a Key(3)
 No immediate effect. If banked with a chest, draw as many bonus cards from the Discard pile as you moved into your Bank.
Sasha's Play Area:
 Key(3)
 Mermaid(5)

Draw again? (y/n): y
Sasha draws a Key(7)
BUST! Sasha loses all cards in play area.
--- Round 1, Turn 2 ---
Marge's turn.
Marge's Bank:
| Score: 0
Marge draws a Key(2)
 No immediate effect. If banked with a chest, draw as many bonus cards from the Discard pile as you moved into your Bank.
Marge's Play Area:
 Key(2)

Draw again? (y/n): y
Marge draws a Hook(5)
 No cards in your Bank. Play continues.
Marge's Play Area:
 Key(2)
 Hook(5)

Draw again? (y/n): y
Marge draws a Mermaid(6)
 No effect but Mermaids are worth more.
Marge's Play Area:
 Key(2)
 Hook(5)
 Mermaid(6)

Draw again? (y/n): n
 Chest and Key activated. Added Key(3), Key(7), Mermaid(5),  to your bank.
Marge's Bank:
 Key(7) Key(3) Key(2)
 Hook(5)
 Mermaid(6) Mermaid(5)
| Score: 18
--- Round 2, Turn 3 ---
Sasha's turn.
Sasha's Bank:
| Score: 0
Sasha draws a Chest(5)
 No immediate effect. If banked with a key, draw as many bonus cards from the Discard pile as you moved into your Bank.
Sasha's Play Area:
 Chest(5)

Draw again? (y/n): y
Sasha draws a Chest(3)
BUST! Sasha loses all cards in play area.
--- Round 2, Turn 4 ---
Marge's turn.
Marge's Bank:
 Key(7) Key(3) Key(2)
 Hook(5)
 Mermaid(6) Mermaid(5)
| Score: 18
Marge draws a Oracle(3)
 The Oracle sees a Hook(3)
Marge's Play Area:
 Oracle(3)

Draw again? (y/n): y
Marge draws a Hook(3)
 Select a highest-value card from any of the suits in your Bank:
 (1) Hook(5)
 (2) Mermaid(6)
 (3) Key(7)
 Which card do you pick? 2
Marge draws a Mermaid(6)
 No effect but Mermaids are worth more.
Marge's Play Area:
 Hook(3)
 Oracle(3)
 Mermaid(6)

Draw again? (y/n): y
Marge draws a Sword(5)
 No cards in other player's Bank. Play continues.
Marge's Play Area:
 Sword(5)
 Hook(3)
 Oracle(3)
 Mermaid(6)

Draw again? (y/n): n
Marge's Bank:
 Key(7) Key(3) Key(2)
 Sword(5)
```

```
 Sword(5)
 Hook(5) Hook(3)
 Oracle(3)
 Mermaid(6) Mermaid(5)
| Score: 26
--- Round 3, Turn 5 ---
Sasha's turn.
Sasha's Bank:
| Score: 0
Sasha draws a Chest(4)
 No immediate effect. If banked with a key, draw as many bonus cards from the Discard pile as you moved into your Bank.
Sasha's Play Area:
 Chest(4)

Draw again? (y/n): y
Sasha draws a Kraken(6)
 Draw 3 cards from the deck and play each:
Sasha draws a Key(6)
 No immediate effect. If banked with a chest, draw as many bonus cards from the Discard pile as you moved into your Bank.
Sasha draws a Sword(7)
 Steal the top card of any suit from the other player's Bank into your Play Area:
 (1) Oracle(3)
 (2) Sword(5)
 (3) Hook(5)
 (4) Mermaid(6)
 (5) Key(7)
 Which card do you pick? 4
Sasha draws a Mermaid(6)
 No effect but Mermaids are worth more.
Sasha draws a Map(3)
 Draw 3 cards from the discard and pick one to add to the play area:
 (1) Chest(5)
 (2) Chest(3)
 Which card do you pick? 1
Sasha draws a Chest(5)
BUST! Sasha loses all cards in play area.
--- Round 3, Turn 6 ---
Marge's turn.
Marge's Bank:
 Key(7) Key(3) Key(2)
 Sword(5)
 Hook(5) Hook(3)
 Oracle(3)
 Mermaid(5)
| Score: 25
Marge draws a Kraken(3)
 Draw 3 cards from the deck and play each:
Marge draws a Kraken(4)
BUST! Marge loses all cards in play area.
--- Round 4, Turn 7 ---
Sasha's turn.
Sasha's Bank:
| Score: 0
Sasha draws a Kraken(5)
 Draw 3 cards from the deck and play each:
Sasha draws a Map(5)
 Draw 3 cards from the discard and pick one to add to the play area:
 (1) Kraken(6)
 (2) Kraken(4)
 (3) Mermaid(6)
 Which card do you pick? 3
Sasha draws a Mermaid(6)
 No effect but Mermaids are worth more.
Sasha draws a Hook(6)
 No cards in your Bank. Play continues.
Sasha draws a Key(5)
 No immediate effect. If banked with a chest, draw as many bonus cards from the Discard pile as you moved into your Bank.
Sasha's Play Area:
 Key(5)
 Hook(6)
 Map(5)
 Mermaid(6)
 Kraken(5)

Draw again? (y/n): n
 Chest and Key activated. Added Key(6), Sword(7), Kraken(6), Kraken(4), Chest(5),  to your bank.
Sasha's Bank:
 Chest(5)
 Key(6) Key(5)
 Sword(7)
 Hook(6)
 Map(5)
 Mermaid(6)
 Kraken(6) Kraken(5) Kraken(4)
| Score: 41
--- Round 4, Turn 8 ---
Marge's turn.
Marge's Bank:
 Key(7) Key(3) Key(2)
 Sword(5)
 Hook(5) Hook(3)
 Oracle(3)
 Mermaid(5)
| Score: 25
Marge draws a Chest(6)
 No immediate effect. If banked with a key, draw as many bonus cards from the Discard pile as you moved into your Bank.
Marge's Play Area:
 Chest(6)

Draw again? (y/n): y
Marge draws a Mermaid(8)
 No effect but Mermaids are worth more.
Marge's Play Area:
 Chest(6)
 Mermaid(8)

Draw again? (y/n): n
Marge's Bank:
 Chest(6)
 Key(7) Key(3) Key(2)
 Sword(5)
 Hook(5) Hook(3)
 Oracle(3)
 Mermaid(8) Mermaid(5)
```

```
| Score: 34
--- Round 5, Turn 9 ---
Sasha's turn.
Sasha's Bank:
 Chest(5)
 Key(6) Key(5)
 Sword(7)
 Hook(6)
 Map(5)
 Mermaid(6)
 Kraken(6) Kraken(5) Kraken(4)
| Score: 41
Sasha draws a Map(7)
 Draw 3 cards from the discard and pick one to add to the play area:
 (1) Chest(3)
 (2) Map(3)
 (3) Kraken(3)
 Which card do you pick? 1
Sasha draws a Chest(3)
 No immediate effect. If banked with a key, draw as many bonus cards from the Discard pile as you moved into your Bank.
Sasha's Play Area:
 Chest(3)
 Map(7)

Draw again? (y/n): y
Sasha draws a Kraken(2)
 Draw 3 cards from the deck and play each:
Sasha draws a Hook(7)
 Select a highest-value card from any of the suits in your Bank:
 (1) Map(5)
 (2) Chest(5)
 (3) Hook(6)
 (4) Mermaid(6)
 (5) Key(6)
 (6) Kraken(6)
 (7) Sword(7)
 Which card do you pick? 4
Sasha draws a Mermaid(6)
 No effect but Mermaids are worth more.
Sasha draws a Oracle(2)
 The Oracle sees a Oracle(4)
Sasha draws a Oracle(4)
BUST! Sasha loses all cards in play area.
--- Round 5, Turn 10 ---
Marge's turn.
Marge's Bank:
 Chest(6)
 Key(7) Key(3) Key(2)
 Sword(5)
 Hook(5) Hook(3)
 Oracle(3)
 Mermaid(8) Mermaid(5)
| Score: 34
Marge draws a Oracle(6)
 The Oracle sees a Hook(4)
Marge's Play Area:
 Oracle(6)

Draw again? (y/n): y
Marge draws a Hook(4)
 Select a highest-value card from any of the suits in your Bank:
 (1) Oracle(3)
 (2) Sword(5)
 (3) Hook(5)
 (4) Chest(6)
 (5) Key(7)
 (6) Mermaid(8)
 Which card do you pick? 2
Marge draws a Sword(5)
 Steal the top card of any suit from the other player's Bank into your Play Area:
 (1) Map(5)
 (2) Chest(5)
 (3) Hook(6)
 (4) Key(6)
 (5) Kraken(6)
 (6) Sword(7)
 Which card do you pick? 4
Marge draws a Key(6)
 No immediate effect. If banked with a chest, draw as many bonus cards from the Discard pile as you moved into your Bank.
Marge's Play Area:
 Key(6)
 Sword(5)
 Hook(4)
 Oracle(6)

Draw again? (y/n): n
 Chest and Key activated. Added Map(7), Oracle(2), Mermaid(6), Chest(3),  to your bank.
Marge's Bank:
 Chest(6) Chest(3)
 Key(7) Key(6) Key(3) Key(2)
 Sword(5)
 Hook(5) Hook(4) Hook(3)
 Oracle(6) Oracle(3) Oracle(2)
 Map(7)
 Mermaid(8) Mermaid(6) Mermaid(5)
| Score: 44
--- Game Over ---Sasha's Bank:
 Chest(5)
 Key(5)
 Sword(7)
 Hook(6)
 Map(5)
 Kraken(6) Kraken(5) Kraken(4)
| Score: 34
Marge's Bank:
 Chest(6) Chest(3)
 Key(7) Key(6) Key(3) Key(2)
 Sword(5)
 Hook(5) Hook(4) Hook(3)
 Oracle(6) Oracle(3) Oracle(2)
 Map(7)
 Mermaid(8) Mermaid(6) Mermaid(5)
| Score: 44
Marge wins!
```